1. Information

Title: Breakers

creator: Leonard Bairoh

2. General description

The player is tasked with completing objectives like defeating the enemy commander to complete these levels/maps. By controlling their team of characters, the player engages in combat against AI-controlled enemies who follow similar restrictions to the player. Maps have tile grids with bonuses, movement restrictions and elevation differences that guide player action. Every character can die permanently, and resources are limited. Characters have different classes and characteristics that separate themselves from one another while also defining the possible weapons and actions the character has access to. The characters grow and equip items based on player choices and fixed factors.

The player sees the field of battle with windows displaying necessary information and the possible actions the player has. Additional tables for character data are displayed when inspecting units.

3. User interface

MAPPING

‘Q’ and ‘E’ rotate the direction which the battle-field is displayd from.

‘Z’ and ‘X’ zoom the direction the battle-field by scaling it up or down.

‘M’ resets the game

WASD controls cursor location, Arrow Keys control camera location,

‘Enter’ is select and ‘Space’ is cancel/select

EXPLAINED

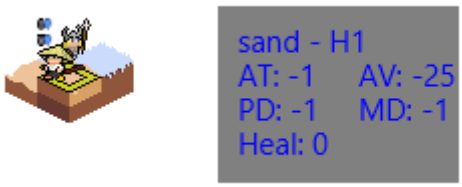
The game does not yet feature a separate start menu, it simply launches when the GUI object is run.

The player interacts with the game using directional inputs and selects actions in menu windows.

By default, the player controls a cursor on the map that moves on the tile grid. When hovering on a character, player or enemy, the player can press a button to

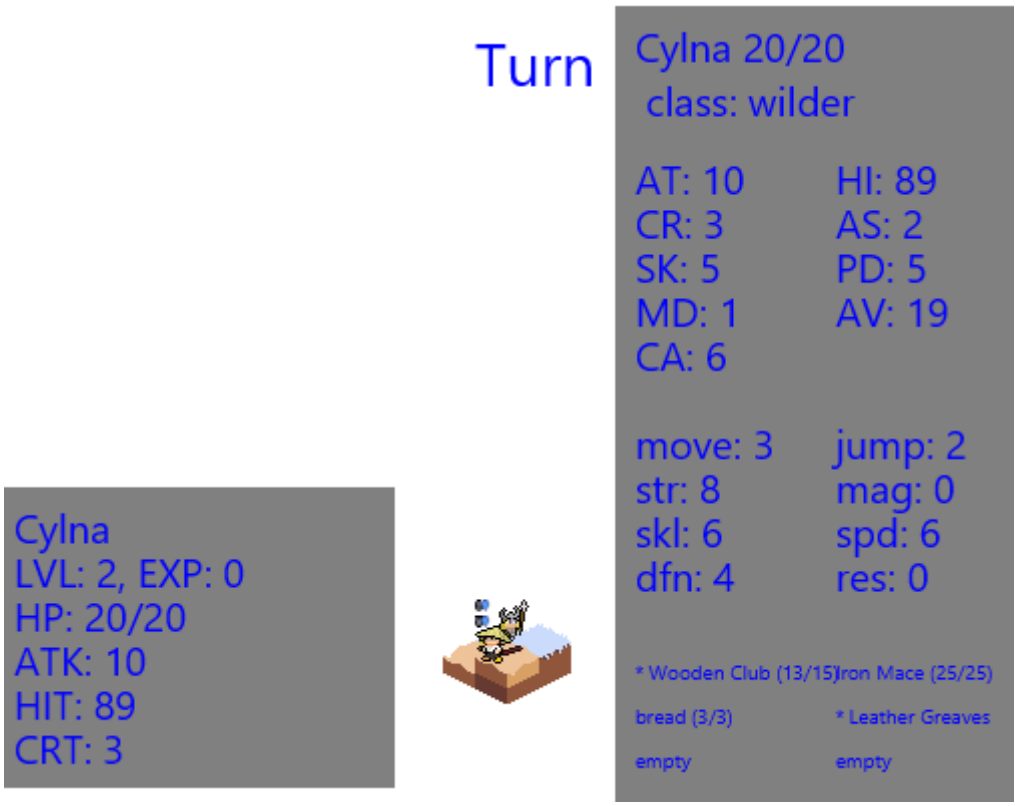
see a window of most info on a character (currently omits some things like the characters weapon ranks).

Whichever tile is hovered over, displays a small window with the tile info.

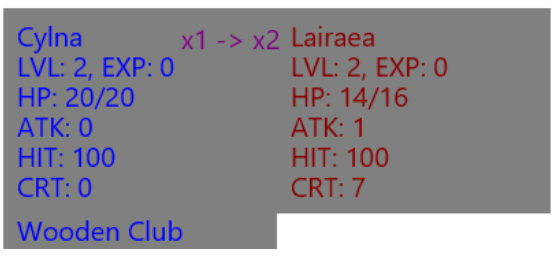


Whoever is hovered over is given a small window of info and colored with the team color (blue for player, green for ally and red for enemy)

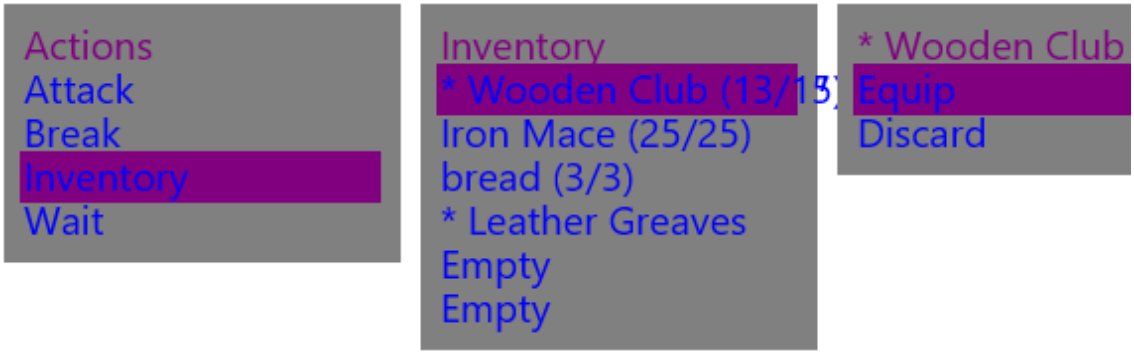
A larger info panel for a hovered character can be displayed with the ‘tab’ button.



Menus open when the player selects a character and from there they can select an action to perform. When selecting a target for the action, a cursor appears under the currently targeted unit.

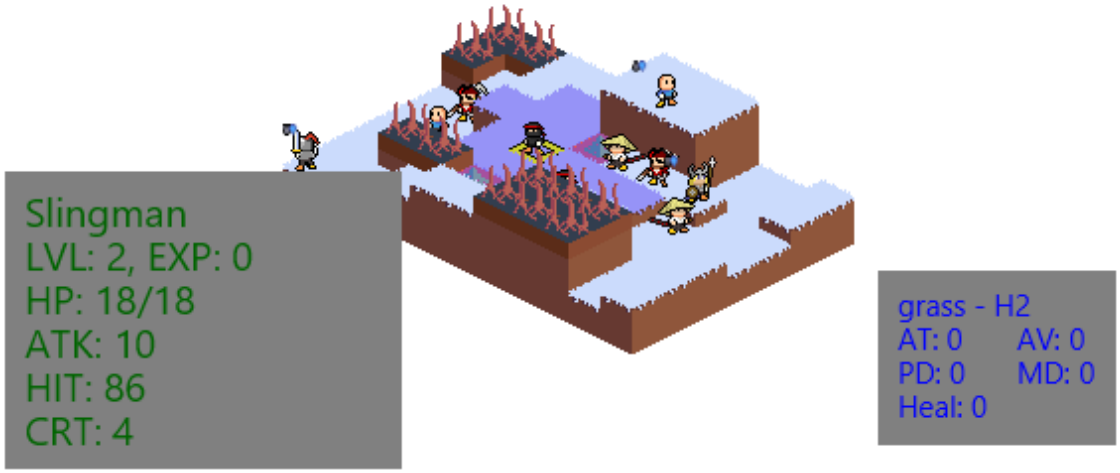


Equipped items are displayed with an asterisk. Selecting equip on equipment toggles its equipped status and unequips another one if it is meant for the same “spot” (having to helmets on at once would be quite silly).



A classes weapon ranks decide which type and rank of weapons a unit can use. For example, a character with ‘D’ in blunt weapons cannot use a Damascus Mace (‘B’) but can use a wooden club (‘E’).

Movement range can be shown for all characters who’ve yet to move during the turn regardless of team if their tile is selected with enter. When the movement is displayed like this, selecting one of the blue tiles with ‘enter’ moves a player character onto it.



Additional information

Maps (battlefields/levels) are grids of tiles that the player characters and enemies interact on. The player and enemy move their characters in turns, on player and enemy phase, where each character can select an action to commit. Confirming the action displays an associated animation and shows the result of the choice. Some actions like equipping do not result in the turns ending.

Characters have stats like strength, defense, speed and weapon rank. These determine the numbers used to calculate the results of initiating combat between two characters that then plays out according to them. Which weapons can be used and the number of tiles a character can move in a turn is determined by class.

When selecting a character to attack the player sees the projected result of interaction. In combat if the range of the attacker and defender are the same then the defender can counterattack. They attack each other in turns and if one has a much higher attack speed they'll attack twice. The attacker always hits first unless the opponent has much higher combat skill.

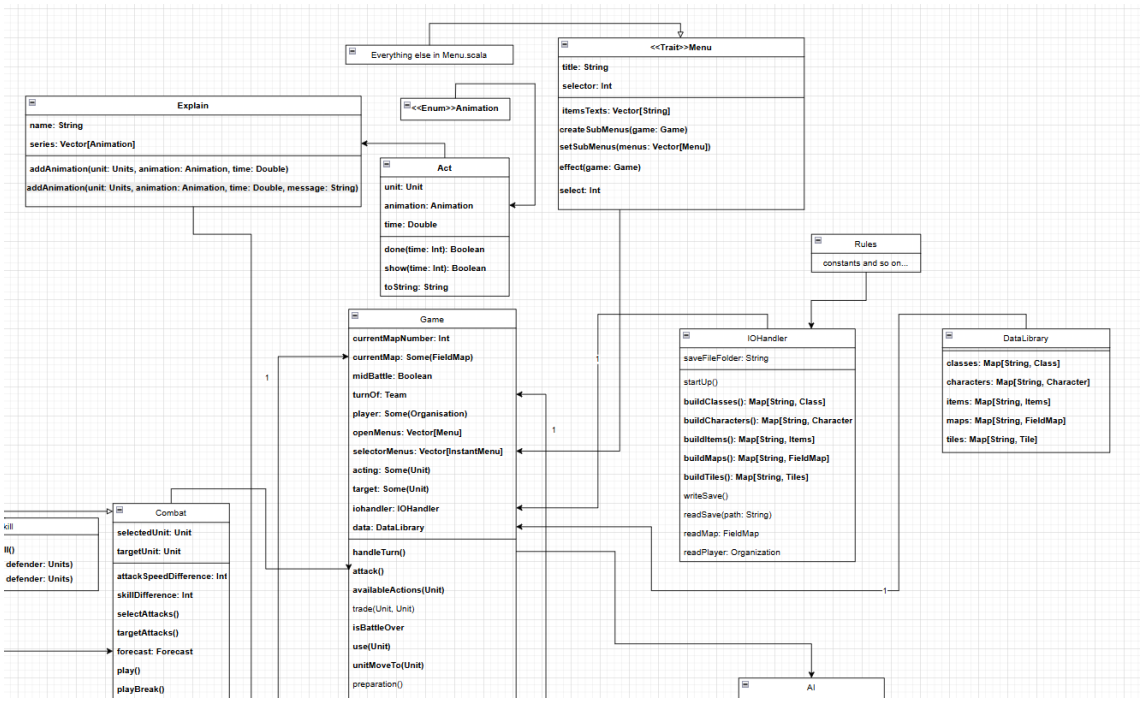
Some classes can fly, some heal, and others are bulky. The classes are not meant to be equal since access to them is limited and other characters will require less investment to use or have more combat utility than others. Weaker characters can be used to replace stronger ones that died to allow the player to still have a chance to continue.

Different classes have different utilities and stats. These stats grow on level-ups when characters gain enough experience points and are randomly given based on growth rates.

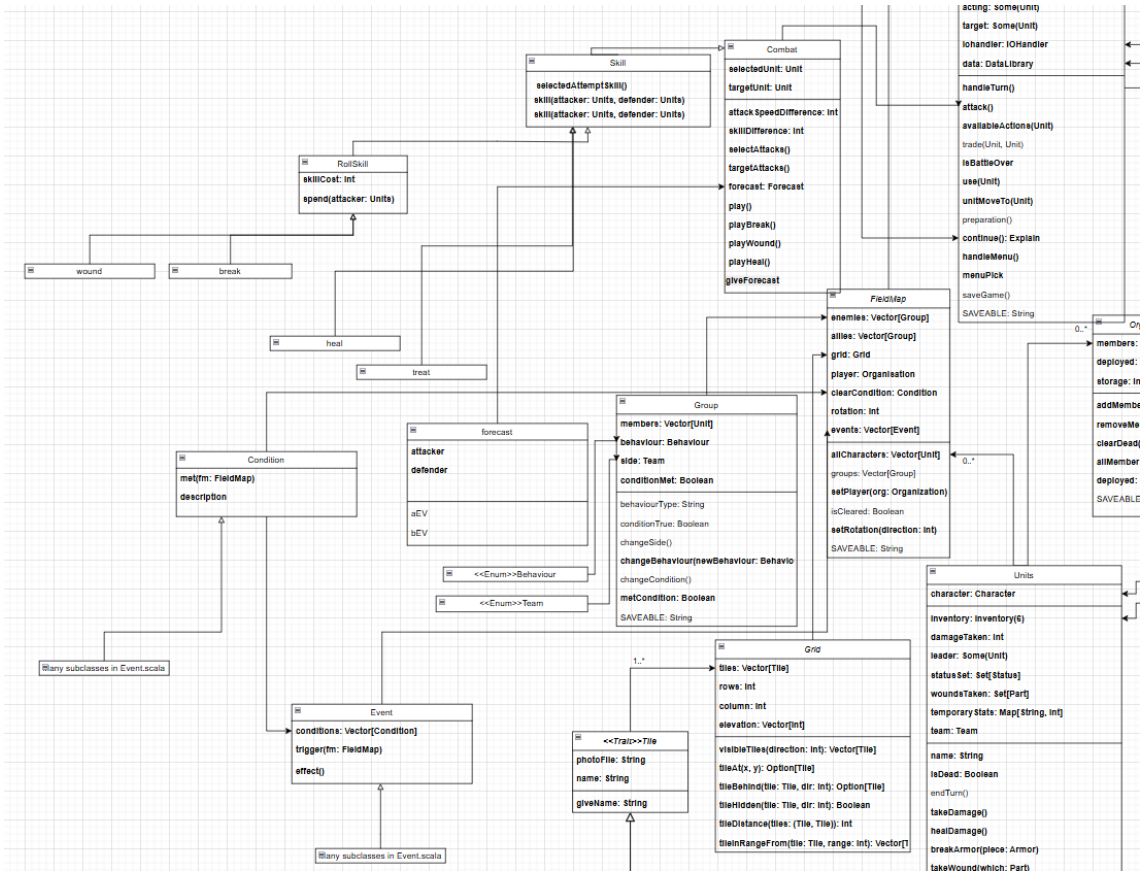
An important mechanic is breaking armor and wounding enemies for strong short-term status ailments. Armor prevents ailments based on body-part.

4. Program structure

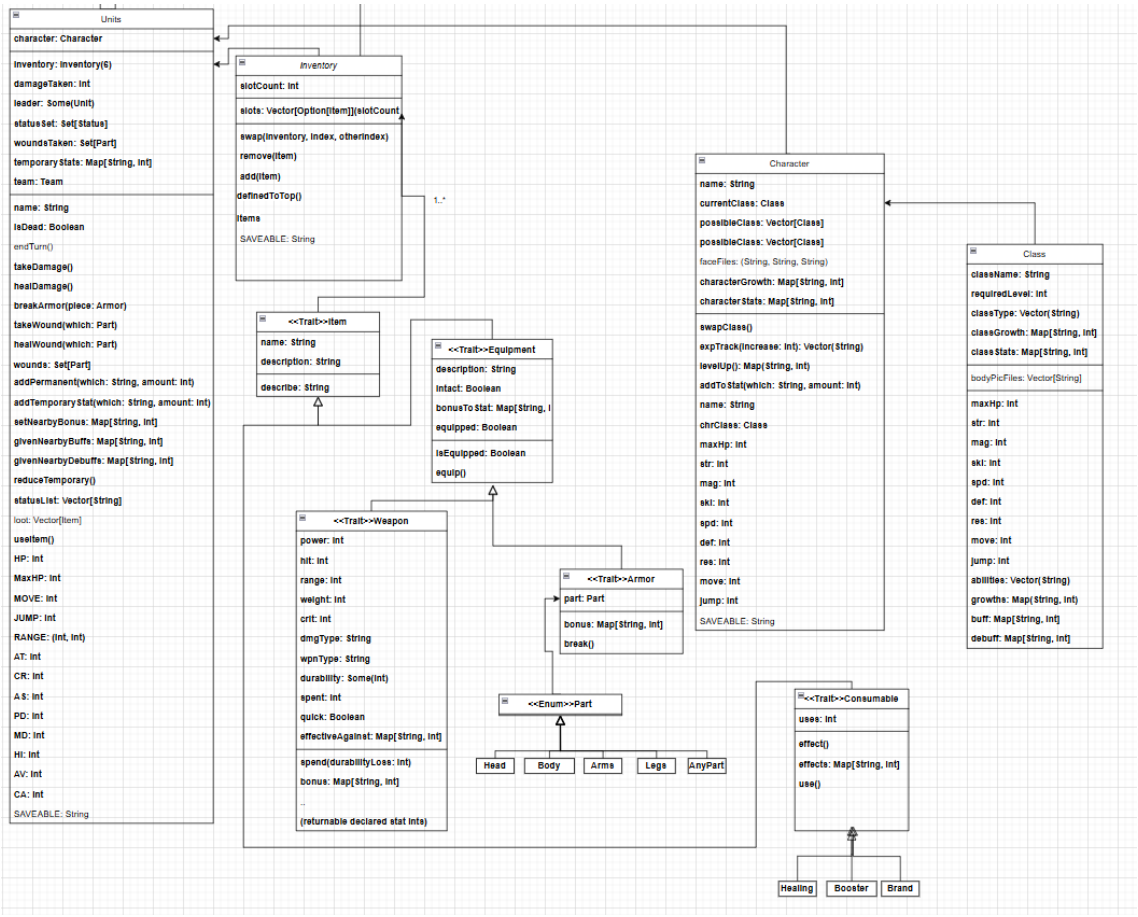
UML OF STRUCTURE



Explains, Acts, Animations, IOHandler, DataLibrary, Games, Menus



Combat, Groups, FieldMaps, Events, Conditions



Units, Characters, Classes, Items, Inventories

The full UML will be an attachment to the document, however even it won't feature all the subclasses for traits like menu or conditions because it would unnecessarily complicate the chart too much.

GUI STRUCTURE

The GUI object handles connecting to a game with startUp() and drawing what is happening. It consists of an AnimationTimer that runs on ticks and updates the game situations or animations. Draws images with methods that represent all the different tiles, units and

MOST IMPORTANT

GUI

- startUp() = Creates the Game instance and initializes it so that the player can start playing.

- drawAllMap(pics: Vector[ImageView], g: GraphicsContext) = draws all the non-menu / non-text images. Sorts them based on with depth and y priority to produce a correctly overlapping result.
- drawMenus (menus: Vector[Menu], g: GraphicsContext) = draws all menus with matching Window components
- handlePress(event: KeyEvent) = handles all key presses

Game

- continue() = *Takes the next action on the stack, plays its effects and return the Explain for it that the UI can use to showcase what took place*.
- addToStack(act: Action) = Add an action to the stack to be performed and explained.
- availableActions(unit: Units): Vector[Action] = Actions available to a given unit
- handleTurn() = Decidec who acts and whether the AI should take over.
- handleMenu() = Handles whatever takes place with menus

EXPLENATIONS FOR STRUCTURE

Menus have their own class to make handling all the cascading sub menus easier. It might be that this could still have been easier with ScalaFX specific components, but the result works fairly well and could easily be used without ScalaFX since GUI can use Game to navigate all these menus and actions with only the WASD, Enter, Space and Tab.

Windows were made to display these menus in a simply way and also to make other easily customizable GUI components with fairly universally applicable use of the ScalaFX library, which makes it easy to change for other graphics libraries.

Game uses a stack for Actions that will be committed next because it makes it easily detachable from the GUI. The GUI takes an explanation for an action, handles it by animating or passing it by and then lets the game continue to handle the next turn. This ensures that every action can be shown and when the showcase is done its easy to simply continue to the next part. This same stack applies to both player and AI actions making reading the actions easy in the user interface.

5. Algorithms

FieldMap

The algorithm that finds tiles for a unit's movement range on the map. The result is based on the subclass of the checked tiles, the movement integer of the unit and their movement type.

Check the unit's occupied tiles neighboring tiles, reduce move by movement cost, and then for each found tile, recursively check the neighboring tiles until movement cost is less than the reduction from the tile. Return the list of the tiles found. If the tile is occupied by a member of the same team, return only neighbors. If the tile is unoccupiable but can be flown over and the unit can fly, return only neighbors. Otherwise, if the tile is not occupiable and unoccupied then don't take the current tile into account for possible tiles.

Game

The algorithm for finding all possible actions for a unit checks the actions available for each movement tile from the movement tile algorithm and then based on a unit's abilities, weapons, med kits and consumables makes a list of all of these target unit, selected tile, action range, target part combinations

AI

The object AI goes through the list of groups of a given team. It then gets the list of units in the group and then sorts through the available actions, makes a list of the best actions and then chooses which one to execute first. From these "best" actions the one picked is determined by selectNextAction where prioritization is skills in the order Treat>Break>Wound and if no skills are available then the unit action with the best estimated value is chosen. This process is repeated until no one in the group can act.

Whereas the "best" action is decided as first of these if the type of action is not empty:

treat highest level target, break for highest hit rate, wound for highest hit rate,

attack with highest estimated value minus one third of enemy estimated value to prioritize high damage inflicted and low damage taken,

heal whoever has taken the most damage, use a consumable item if hp is not max,

wait if nothing else can be done

6. Data structures

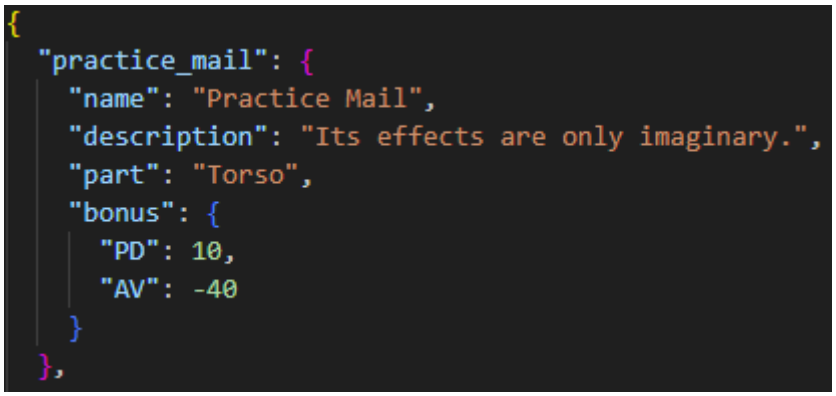
The project uses mostly Vector, Map, Seq and Tuples to store information. These are mostly immutables since I tried to avoid over using mutables but this lead to

many var vector = vector.appended(item) situations which would probably be better as simple mutables. Some methods were implemented with mutables as collectors when other structures would have taken more time and effort for diminishing returns like in levelUp() in character. Therefore there are some mutable.Buffer and mutable.Map uses as collectors, but there aren't really any used for variables or constants since they can just be variables.

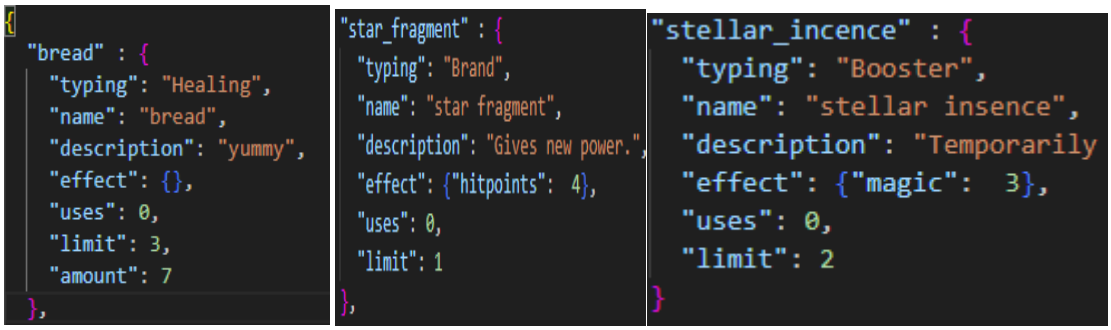
7. Files and Internet access

File reading is all done from JSON-files which are stored in the data folder of resources. These files are read with upickle as maps that are used to create instances of classes for the data library that can then by copied to make inventories out of items or units out of characters and classes.

Items



Items of the subclass Armor are stored in armors.json, where they are given a specified name, description and part as strings. The bonuses bestowed by the armor are given as map of string to int. If the given part is not found from the body parts specified in rules, it is made as an AnyPart armor and won't protect from any wounds.



Non-equipment items, consumables are divided into three different categories that change the way "effects" affect the unit that uses it based on the defined map

of string to int. “typing” must be one of these three categories of “Healing”, “Booster” and ”Brand”. The item’s spent value is based on “uses” and its limit is set to by “limit”.

```
"dagger": {
  "name": "Dagger",
  "description": "A simple dagger.
  "rank": "E",
  "spent": 0,
  "durability": 25,
  "quick": false,
  "wpntype": "Sharp",
  "dmgtype": "force",
  "power": 1,
  "hit": 70,
  "crit": 10,
  "range": [1, 1],
  "weight": 1,
  "effective": {},
  "bonus": {}
},
```

```
{
  "simple_medkit": {
    "name": "Simple Medkit",
    "description": "A simple medical packet
    "durability": 15,
    "heal": 6,
    "range": [1,1],
    "spent": 0,
    "bonus": {}
  },
```

Weapons share the same requirements as other equipment in addition to all those needed to make a weapon to create an instance of the WeaponFile case class. “wpntype” can define any string as weapon type so a user can add new ones that could also match new ranks set for classes. This also makes it possible to add new weapon relations to Rules.

For med kits in the medkits.json file, the info is given similarly to weapon where it has durability and range, but instead of damage and other combat values it has “heal”.

Unit requirements

Classes (as in a characters job) are defined in classes.json where the new instance can be made directly with the parameters from the objects in classes.json.

Characters read from characters.json require the data library to already contain their classes since when creating a character their list of classes is given as a list of strings for their names which is then mapped to the data libraries map of classes to give the proper classes. Otherwise, CharacterHandler is quite straight forward in reading parameters for the values.

```
"inventory_Cylna": [
  ["w_club",2],
  ["i_mace",0  ],
  ["bread",1],
  ["leather_greaves",0]
],
```

Inventories are stored as “inventory_” plus the characters name. Within them, items are stored as a pair of the item name key and an int representing their spent/used amount which is then deducted when creating the inventory to give correct durability left.

```
"1": {
  "enemies": [ [["BossLairaea",2,[1,1]]] ],
  "allies": [[]],
  "joining": [ ["Cylna",0,[0,0]] ],
  "events": {},
  "grid": {
    "tiles": ["gray_field","gray_field",
              "sand_field","sand_field"],
    "row": 2,
    "column": 2,
    "elevation": [0,1,0,1]
  },
  "clear": {
    "title": "kill",
    "target": ["Lairaea"]
  },
  "lose": [],
  "deploy": [[0,0],[0,1],[1,0]],
  "rotation": 2,
  "turnNumber": 1
},
```

```
"events": {
  "aynia_joins": {
    "title": "reinforcement",
    "units": [
      ["Aynia",[4,5]]
    ],
    "team": "Player",
    "turns": [2]
  },
  "knight_appears": {
    "title": "reinforcement",
    "units": [
      ["Lochagos",[1,1]]
    ],
    "team": "Enemy",
    "turns": [4]
  }
},
```

FieldMaps

With the necessary data to create units and grids, FieldMaps can be made. These are the most complex files read. MapHandler creates groups from arrays of (character name, damage taken, position on map). These groups are either “enemies” or “allies” which sets what team the group is fighting for. “joining” defines who is joining the player team at the beginning of the map in a similar way.

The grid is made based on the names given for they keys of tiles in the data library. The clear condition is read from “clear” which defines based on title what kind of other keys it should also take into account. “deploy” defines which spots those already in the player organization will occupy at the start of the map. “events” can currently only define new Reinforcement events, where a unit is added at the given position when the turn count matches any of the “turns”.

8. Testing

Testing a first

As outlined in the project plan, most testing for the first few weeks was done by reading command line messages within the created Test scala class. This was done for simple component classes like inventory and items, and tiles and grids. As the game got more complex however, a test GUI environment was made to facilitate testing combat playback and map building. This was a few weeks sooner than starting the proper GUI class since some testing was unexpectedly slow in only text like proper movement ranges.

Testing with GUI

Most testing after making the proper final GUI class was done based on visual and debugger information. This was the easier part to test since most components were already working and if not, it was easily visible. Testing the proper display and function of menus was the most time-consuming part of this phase.

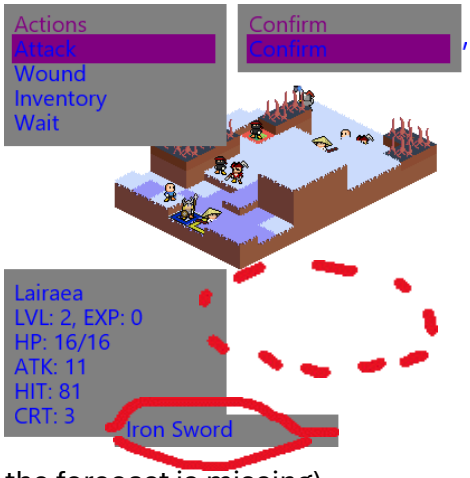
Unit testing

File reading is tested by using specific unit tests for designated test subclasses. For the test map the grid positions and tile names are tested and so is the clear condition. Most importantly things that would be difficult or time consuming to test in GUI are tested here like copying from the data library. When, for example, the inventory for a unit is read, it is important that for soldier 1 and soldier 2 they are separate as in their actions like throwing a weapon away won't make them both lose their weapons. It is also tested that those inventories refer to separate items in their slots.

9. Known bugs and missing features

BUGS

When going back in menus from combat selects, the selector menus and forecast menus can be bugged and appear prematurely.



(In this picture the selector menu is visible but the forecast is missing)

When reinforcements appear on an occupied tile the game adds the group to the map without placing them on the map causing their turns to never happen leaving the AI stuck in an infinite loop. It should be fixed by preventing an unplaceable group or its specific units from being added in the effect method of Reinforcement. Giving a unit in a map file an out of bounds, occupied or unoccupiable tile might also trigger the same problem.

ISSUES

Many variables that should/could be private or not set to as such because of lack of time to thoroughly check through them all again. Most were first set as public to make testing values faster but now leaves the code to be more unclear and unsafe than it should be.

File reading is still quite finnicky and could cause problems fairly easily if more complex changes are made. Giving incorrect positions or dimension might break FieldMap. In the trait ReadingHandler, the default value should be an empty file reading instead of the reading for tiles.json, because now removing that will make all other file reading fail.

Icons for status effects appear and disappear instantly in combat instead of when the attack connect or lands, because the event has already taken place when the animations are playing.

Zooming too far flips the map because it changes the scaling factor and the drawing order is incorrect.

MISSING

AI

selectNextAction doesn't prioritize healing properly since healing doen't have its own estimated value calculation but instead uses the regular combat attack power calculation.

The AIs use of different weapons in a characters inventory might still have untested problems.

File reading

Adding new statuses with files or status effects to weapons.

Methods for reading groups to create FieldMaps can't handle behavior types. All groups are set by default to aggressive.

Reading lose conditions currently just defaults to Route player. It should be simple enough to add with the structure used for clear conditions with some time.

No way to change the images displayed for a class or character within the .json files.

Trading

Many methods for trading are already implemented in the code, but the menu logic required for it has not been done yet due to time constraints.

More information for player

Things like item descriptions are currently only visible in the files and code themselves as there is no way to "inspect" an item yet. This could be done by providing a string describing the currently selected submenu for example to the GUI. The same goes for things like a class's types and weapon ranks which are currently invisible but work correctly.

Portrait

The portrait class for handling a unit's images in a succinct way is completely unimplemented and is meant to replace the current suspect and problematic unit image system that matches strings in maps within GUI.

Settings

Settings that can be changed during run time and saved in a config file.

Errors

Stricter, more thorough, and informative error handling.

Battle preparations

Having no map preparation screen even though game contains some values required for tracking it, makes it impossible to manage the player organization inventory or deployment if player character count exceeds the deployment slots of the next map.

10.

Strengths

Extensive customizability with file reading for characters, their classes, inventories and weapons combined with robust game mechanics makes adding new levels and units to the game relatively fun and streamlined.

Simple rule changes in Rules to change internal game logic making it easy to tweak and balance. Changing and adding mechanics within classes doesn't seem too daunting of a task with the current component set.

Threatening AI capable of attacking the most vulnerable opponent, using skills to wound or break dangerous targets, healing and treating the wounds of one's own units, using items for healing and waiting in case there is nothing else to be done in movement range. The AI does not however consider different tiles or nearby bonuses so the player can trick it into poor positioning like standing in sand or tar.

Weaknesses

More thorough and flexible unit testing would make it easier to notice flaws if someone else were to edit the code or the data files.

The way images are handled, especially for the units still leaves much to be desired as the plan is to make a system that incorporates class, character faces and direction, but now leaves much to be desired. Status effects other than wounds and armor can't be displayed since if are not already defined in GUI which is counterintuitive and cumbersome.

There are many unimplemented or dead end methods, code, and thinking that might make it difficult to understand for others. Committing and proper commit messages might also have helped but they are somewhat lacking, making the code quite a bit more complicated to understand than it should be.

11.

The initial estimated plan matched the first two weeks of implementing the basic components, but after that most of the time estimations were doubled from the original.

After this I made started making the environment for testing with graphics which took way longer than I expected, but eventually I was able to make it work, but some of the problems like matching images from maps persist in the final product.

The next two weeks were used for getting the grid and combat to function properly together. All commands to the graphical interface were made with text commands (as were the animation) in a match case for input strings which was luckily refactored for the final product.

Then making the AI took quite a bit of time because testing it without proper turn handling in game it would not function properly. Testing the AI also took a lot of time, but in this case it was about as much as I expected.

Another two week period was used for starting the proper GUI with menus and animation. Implementing menus was another unexpected time sink, but not as much as the following week of file reading.

Lastly, in the final two weeks I tried to polish out bugs and implemented unit testing.

12.

I would say that there are more shortcomings in the document than in the project game itself. It mostly works even though there are still a few kinks to iron out. The biggest problems come from uneven error-handling when reading files, but with more time I'd like to have tested saving with JSON-files and path finding for the AI. Despite this I find the gameplay and systems to be fun which is why I'd like to continue developing the project in the future. It was really nice to make a project that is inspired by and tries to iterate on my personal favorite genre of games, and I learned a lot about time management and planning for long, extensive projects.

13.

Code was not copied or edited from elsewhere except for the course examples for file reading and unit testing which I was using as a base when implementing those features. No AI tools were used in making this.

Inspiration taken from The **Armies** of Islam **7th–11th** Centuries

All art is either custom-made by me or from this open game art project.

https://opengameart.org/content/isometric-classic-hero-tiles-32x32